

How Our Graph Transformer Reads a Graph

Why the tokenization keeps failing, and what the failures have in common

connectednesshard · 2026-06-23

1 The job a transformer has to do

A transformer does not understand graphs. It understands **sequences of vectors** — “tokens”. Every attention layer takes a set of tokens and lets each one look at all the others. So before any learning can happen, we must answer one question:

How do we turn a graph into a list of tokens?

This single choice — the **tokenization** — decides everything. Get it wrong and no learning rate, depth, or epoch count will save you. Everything we have struggled with this week lives inside this one choice.

1.1 The task we are testing on

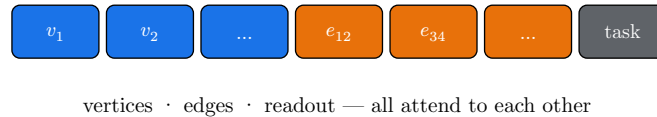
Our hard connectivity dataset is built so that **only global reasoning** can solve it. Every graph is two dense blobs. The two classes are made deliberately indistinguishable to any local shortcut:



Both classes have the **same number of edges** and the **same degree distribution**. The only difference is whether one edge happens to cross between the blobs. To see that, the model must trace reachability across the whole graph. That is the point — it is a pure test of global reasoning.

2 Attempt 1 — vertices + edges + a task token

This is the Sanford-style tokenization in `src/transformer.py`. We build one token per vertex, one token per edge, and a single “task” token that reads out the answer:



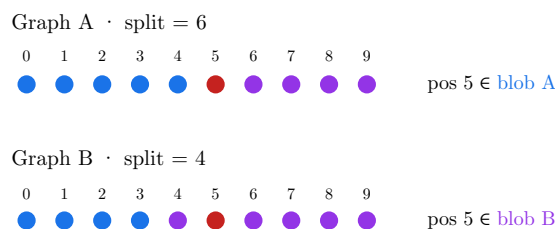
An edge token has to “name” the two vertices it connects. We do that by giving each vertex an **identity vector** and building the edge token from its two endpoints’ identities:

$$\text{edge}_{uv} = W(\text{id}_u + \text{id}_v)$$

Here is where it breaks. The identities come from a **shared lookup table** indexed by node position: `nn.Embedding(max_nodes, 16)`. Position 5 in **every** graph reads the same row of that table.

2.1 Why it gets stuck at $\ln 2 \approx 0.693$

The blob split point varies from graph to graph. So node position 5 lands in blob A in one graph and blob B in another:



When we train, Graph A pushes the embedding of “position 5” to mean **blob A**, and Graph B pushes the **same** embedding to mean **blob B**. The two gradients point in opposite directions and **cancel**:

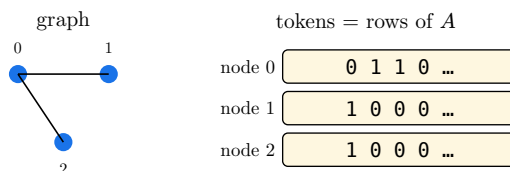
$$\nabla_{\text{id}_5} = \underbrace{g_A}_{\text{“be blob A”}} + \underbrace{g_B}_{\text{“be blob B”}} \approx 0$$

The optimizer receives almost no net signal. The model parks at the only safe answer — output $[0.5, 0.5]$ for everything — whose loss is exactly $\ln 2 = 0.6931$. That is the flat line we watched for 320 epochs. Raising the learning rate did nothing, because $\text{lr} \times 0 = 0$.

The tell: it overfits 10 graphs perfectly but cannot move on 1000. With 10 fixed graphs each position has **one** consistent role, so nothing cancels. Add variety and the shared table tears itself apart.

3 Attempt 2 — adjacency rows as tokens

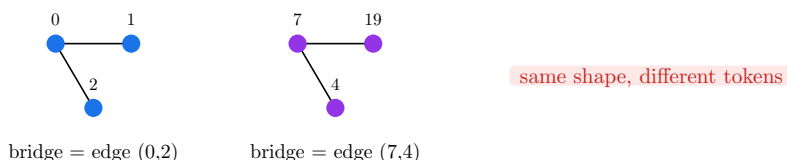
Idea: skip the identity table entirely. Let each node’s token **be** its row of the adjacency matrix — a binary vector saying exactly who it connects to (padded to `max_nodes = 24`).



This **fixed the gradient problem** — the loss fell to 0.03, the model clearly learns. But test accuracy stalled around **0.59**. It **memorizes** and refuses to **generalize**. Why?

3.1 Why it cannot transfer

Row i has a 1 in column j purely because of how this particular graph happened to **number** its nodes. The very same graph, relabelled, produces completely different tokens:



The network learned “node 3 has a 1 in column 15” — a fact glued to one arbitrary numbering. A new test graph numbers its nodes differently, so none of the learned patterns line up. The information is **there**, but in a form that does not survive relabelling. This property has a name: the representation is **not permutation-invariant**.

4 The one problem behind both failures

Tokenization	Trains?	Generalizes?	Failure
vertices + edges + learned IDs	no	—	shared ID table → gradients cancel → stuck at $\ln 2$
adjacency rows	yes	no	tokens tied to node numbering → no transfer

Both are two sides of the same coin — **node identity**. A transformer must be able to tell its tokens apart, but for a graph the “names” of nodes are arbitrary. We need identities that are:

unique within a graph
stable across epochs

so an edge token can name its two endpoints,
so the gradient does not fight itself,

not shared across graphs so position 5 in one graph cannot collide with position 5 in another,
structure-only so a relabelled copy is treated identically.

The learned table violates **3** (shared across graphs). The adjacency row violates **4** (carries the arbitrary numbering). No tokenization we have tried satisfies all four at once — and that, not depth or learning rate, is why the model will not learn.

4.1 The direction that satisfies all four

Give each node a **fresh random identity, drawn once when the graph is created and stored with it** (`data.nid` in the dataset, not a lookup table). Random vectors are **unique** with probability 1, **stable** because they are saved, **never shared** because each graph rolls its own, and **structure-only** because they carry no positional meaning — they are pure “name tags” the attention layers can use to bind edges to vertices. This is the standard fix in the graph-transformer literature, and it is the natural next experiment.